

QA Manager Visibility + JK6B Defect Chain

Two governance audits in one read-only sweep. Q1: why does ADMIN see 7 Site Visits while QA Manager sees 0? Q2: for visit JK6B (PE JALAN KINRARA 6B) showing “1 defect”, was a Defect row actually created, and why does Operations Board return 0 matching defects?

HEADLINE — (1) The QA Manager visibility gap is real and code-level. UserRole has no QA persona; QA Manager has role MANAGER. All four read access scopes drop non-ADMIN to team-membership only. The QA-actor concept (isQaActor) exists but is consulted only by assertCanGovernQa for verify/close authority — not by any read path. (2) The Site Visits “defectsFound” count is sourced from InspectionItemResult rows with isDefect=true, NOT from Defect rows. A flagged result can exist without a Defect row, because Defect rows are materialised lazily by Dashboard/Operations Board calls scoped to the viewer.

1. Why does ADMIN see 7 Site Visits while QA Manager sees 0?

1.1 The role enum

UserRole (prisma/schema.prisma:10–17) is ADMIN | MANAGER | SUPERVISOR | TECHNICIAN | VIEWER | CLIENT. There is no QA_VALIDATOR, QA_SUPERVISOR, or “QA Manager” role. A user described as “QA Manager” in product language is, in code terms, a MANAGER with optional QA_VALIDATION capability and ASCURE-organization membership — the same shape isQaActor tests.

1.2 The four access scopes

Endpoint	Helper	ADMIN behaviour	Non-ADMIN behaviour
GET /site-visits (list)	site-visits.service.ts:2136–2151 accessScope	returns {} — no extra filter	returns { team: { members: { some: { userId: user.id, isActive: true } } } }
GET /dashboard	dashboard.service.ts:683–696 siteVisitAccessScope	returns {}	returns the same team-membership filter
GET /dashboard (inspection / defect counts)	dashboard.service.ts:664–681 inspectionAccessScope	returns {}	returns { siteVisit: { team: { members: { some: ...user.id... } } } }
GET /defects/operation-s-board	defects.service.ts:3144–3161 inspectionAccessScope	returns {}	returns the same nested team-membership filter on inspection.siteVisit

1.3 Where the QA-actor concept lives — and where it doesn't

apps/api/src/common/authorization/qa-actor.ts defines two helpers:

```
export function hasQaActorShape(user) { ... }  
  // user.organization.type === ASCURE &&  
  // user.organization.isActive === true &&  
  // user.capabilityAssignments.length > 0 (active QA_VALIDATION)  
  
export async function isQaActor(prisma, user) { ... }
```

Repo-wide grep for callers of these helpers returns **one and only one**: defects.service.ts:3174 inside assertCanGovernQa. That method gates **verify / close authority for defects**. None of the four

read scopes above consults `isQaActor`.

1.4 Is QA Manager incorrectly restricted to team membership?

Yes. The pilot governance (V10 §MAINHEAD Visibility Algorithm step 7; V8 §15) requires QA to have cross-MAINHEAD, cross-team read visibility. The implementation today only honours that contract by giving QA users the `ADMIN` role — which is over-privileged — or by adding them as a team member to every operational team — which is operationally impractical. With role `MANAGER` (the typical pilot setup), QA sees only visits whose team they belong to. For visits executed by inspection contractors, that count is zero.

The minimal correct contract would be: each read access scope checks `isQaActor (user)` in addition to the `ADMIN` check, and returns `{}` (or a region-scoped equivalent) for QA actors. No code change is being proposed here — assessment only.

2. JK6B — “1 defect” chain

2.1 Where the “1 defect” count comes from

Source: `SiteVisitsService.getRollups` (`site-visits.service.ts:1156–1286`). The Site Visits list page computes the `defectsFound` column by counting `InspectionItemResult` rows with `isDefect=true` belonging to inspections of the visit. It does NOT count Defect rows.

```
prisma.inspectionItemResult.findMany({
  where: { isDefect: true, inspection: { siteVisitId: { in: siteVisitIds } } },
  select: { inspection: { select: { siteVisitId: true } } },
})
// then: defectsByVisitId.set(siteVisitId, (count) + 1) for each row
```

2.2 Where Defect rows come from (lazy materialisation)

`DefectsService.ensureDefectsForAccessibleItems` (`defects.service.ts:1426–1459`) is called **only** by the Dashboard (line 36) and Operations Board (line 567). It scans `InspectionItemResult` rows with `isDefect=true` belonging to inspections the CURRENT VIEWER can access (i.e., filtered by `inspectionAccessScope(user)`) and inserts a Defect row for each via `createMany({ skipDuplicates: true })`.

- Defect rows are **not** created at inspection-submission time.
- Defect rows are **not** created by viewing the Site Visits list.
- Defect rows materialise on-demand the first time an ADMIN (or, in production, a future QA actor with the same access) opens the Dashboard or Operations Board.
- If a QA Manager (role=MANAGER, no team membership) opens Operations Board first, `ensureDefectsForAccessibleItems` runs with their scope — which is empty (no accessible inspections) — and creates **no** Defect rows. The visit’s “1 defect” count on the Site Visits page persists because that rollup counts `InspectionItemResult`, not Defect.

2.3 The two-table mismatch — table by table

Surface	Source table	Filter
Site Visits list — <code>defectsFound</code> column	<code>InspectionItemResult</code>	<code>isDefect=true AND inspection.siteVisitId IN (visit ids)</code> . No tenant or access filter here because the outer list query has already filtered visits by access scope.
Dashboard — defect groupBy counts	<code>Defect</code>	<code>inspectionItemResult.isDefect=true AND inspection.tenantId=user.tenantId AND inspection.siteVisit.team.members.some.userId=user.id</code> (non-ADMIN only)
Operations Board — items	<code>Defect</code>	Same as above. Plus optional query filters (mainhead, severity, status, etc.).

2.4 Determination — Defect row for JK6B

The connected DB does not contain JK6B. Probe across `pencawangCode`, `pencawangName`, substation code/name, and the legacy `SiteVisit.mainhead` text column returned 0 matches. Production access is not available to this assessor. The deterministic answer therefore comes from code analysis plus the two-table mismatch above:

Production state	What the user sees	Why
Defect row does not exist for JK6B InspectionItemResult	Site Visits list shows “1 defect”. Operations Board returns 0 matching defects.	The Site Visits rollup counts the InspectionItemResult row. The Operations Board defect .findMany returns 0 because no Defect row exists yet. ensureDefectsForAccessibleItems has not materialised one — most likely because the only person who opened Operations Board was a non-team-member viewer (QA Manager) whose access scope returned 0 accessible items.
Defect row does exist for JK6B	Site Visits list shows “1 defect”. Operations Board:	
ADMIN view	Should show 1 (in JK6B's MAINHEAD column)	ADMIN access scope is empty; no row-level filter drops the defect. If admin still sees 0, an optional query filter is restricting the page (mainhead, severity, status) — check the URL.
QA Manager view	Returns 0	Non-ADMIN team-membership scope drops the defect because QA Manager is not a member of JK6B's site-visit team. Same root cause as Q1.

3. Production probe to determine which case applies

Run this read-only probe with DATABASE_URL pointed at production. The output answers Q2 unambiguously.

```
// 1) Locate the JK6B visit
const visit = await prisma.siteVisit.findFirst({
  where: {
    OR: [
      { pencawangCode: { contains: 'JK6B', mode: 'insensitive' } },
      { pencawangName: { contains: 'KINRARA', mode: 'insensitive' } },
      { substation: { code: { contains: 'JK6B', mode: 'insensitive' } } },
    ],
  },
  select: { id: true, tenantId: true, teamId: true, status: true,
    substation: { select: { code: true, name: true } } },
});

// 2) Walk the chain: Inspection -> InspectionItemResult -> Defect
const inspections = await prisma.inspection.findMany({
  where: { siteVisitId: visit.id },
  select: {
    id: true, completionStatus: true, submittedAt: true,
    itemResults: {
      select: { id: true, label: true, isDefect: true, severity: true,
        defect: { select: { id: true, status: true, lifecycleStatus: true } } },
    },
  },
});

// 3) Count Defect rows linked to that visit
const defectCount = await prisma.defect.count({
  where: { inspectionItemResult: { inspection: { siteVisitId: visit.id } } },
});
```

- If `itemResults.some(r => r.isDefect)` is true and `defectCount === 0`, this matches Case A in §2.4: the flagged result exists but the Defect row was never materialised. Have an ADMIN open Operations Board once — the next `ensureDefectsForAccessibleItems` call creates the row.
- If both counts are positive but Operations Board for ADMIN still returns 0, check the admin web URL for a `?mainhead=/?severity=?status=` query parameter that may be filtering it out.
- If the flagged item itself does not exist on the inspection (no `itemResult` with `isDefect=true`), then the Site Visits list rollup should not show “1 defect” — that would point to a stale cached page or a data-drift between count and display.

4. Bottom line

- **Q1 answer:** ADMIN sees 7 visits, QA Manager sees 0, because every read access scope (`accessScope`, `siteVisitAccessScope`, `inspectionAccessScope`) returns `{}` for ADMIN and a team-membership filter for every other role. The QA-actor concept exists in code but is consulted only for verify/close authority — not for read visibility. Yes, QA Manager is incorrectly restricted to team membership. This is a governance gap, not a configuration error.
- **Q2 answer:** The “1 defect” count on the Site Visits list is sourced from `InspectionItemResult` rows where `isDefect=true`, NOT from Defect rows. A flagged result can exist without a Defect row because materialisation is lazy and scoped to the viewer. The Operations Board returns 0 either because the Defect row was never created (no ADMIN viewer triggered `ensureDefectsForAccessibleItems`) or because the current viewer is non-ADMIN and the team-scope drops it. Run the §3 probe to confirm which.